

Overview of Sparse attention transformer

Subject: Sparse attention transformer

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

1.

$$\text{MultiQueryAttention}(Q, K, V) = \text{Concat}_{i \in [n \text{ heads}]} (\text{Attention}(XW_i^Q, XW_i^K, XW_i^V)) W^O$$
$$\{\text{MultiQueryAttention}\}(Q, K, V) = \{\text{Concat}\}_{i \in [n \text{ heads}]} \left(\{\text{Attention}\}(XW_{i \in [n \text{ heads}]}^Q, XW_{i \in [n \text{ heads}]}^K, XW_{i \in [n \text{ heads}]}^V) \right) W^O$$

2.

Tokenizers, which convert text into tokens. Embedding layer, which converts tokens and positions of the tokens into vector representations. Transformer layers, which carry out repeated transformations on the vector representations, extracting more and more linguistic information. These consist of alternating attention and feedforward layers. There are two major types of transformer layers: encoder layers and decoder layers, with further variants. Un-embedding layer, which converts the final vector representations back to a probability distribution over the tokens. The following description follows exactly the transformer as described in the original paper. There are variants, described in the following section. By convention, we write all vectors as row vectors. For example, pushing a vector through a linear layer means multiplying it by a weight matrix on the right, as xW .

3.

The feedforward network (FFN) modules in a transformer are 2-layered multilayer perceptrons: $\text{FFN}(x) = \phi(xW^{(1)} + b^{(1)})W^{(2)} + b^{(2)}$ where $W^{(1)}$ and $W^{(2)}$ are weight matrices and $b^{(1)}$ and $b^{(2)}$ are bias vectors, and ϕ is its activation function. The original transformer used ReLU activation. The number of neurons in the middle layer is called intermediate size (GPT), filter size (BERT), or feedforward size (BERT). It is typically larger than the embedding size. For example, in both GPT-2 series and BERT series, the intermediate size of a model is 4 times its embedding size: $d_{\text{ffn}} = 4d_{\text{emb}}$.

4.

Alternative activation functions The original transformer uses ReLU activation function. Other activation functions were developed. The Llama series and PaLM used SwiGLU; both GPT-1 and BERT used GELU. Alternative activation functions are often used in combination with Gated Linear Units in the feedforward module.

5.

The attention mechanism used in the transformer architecture are scaled dot-product attention units. For each unit, the transformer model learns three weight matrices: the query weights W^Q , the key weights W^K , and the value weights W^V . The module takes three sequences, a query sequence, a key sequence, and a value sequence. The query sequence is a sequence of length n_{seq} , query $\ell_{\text{seq, query}}$, and each entry is a vector of dimension $d_{\text{emb, query}}$. Similarly for the key and value sequences. For each vector x_i in the query sequence, it is multiplied by a matrix W^Q to produce a query vector $q_i = x_i W^Q$. The matrix of all query vectors is the query matrix: $Q = X_{\text{query}} W^Q$. Similarly, we construct the key matrix $K = X_{\text{key}} W^K$ and the value matrix $V = X_{\text{value}} W^V$. It is usually the case that all W^Q, W^K, W^V are square matrices, meaning $d_{\text{emb, query}} = d_{\text{query}}$, etc. Attention weights are calculated using the query and key vectors: the attention weight a_{ij} from token i to token j is the dot product between q_i and k_j . The attention weights are divided by the square root of the dimension of the key vectors, $\sqrt{d_k}$, which stabilizes gradients during training, and passed through a softmax which normalizes the weights. The fact that W^Q and W^K are different matrices allows attention to be non-symmetric: if token i attends to token j (i.e. $q_i \cdot k_j$ is large), this does not necessarily mean that token j will attend to token i (i.e. $q_j \cdot k_i$ could be small). The output of the attention unit for token i is the weighted sum of the value vectors of all tokens, weighted by a_{ij} , the attention from token i to each token. The attention calculation for all tokens can be expressed as one large matrix calculation using the softmax function, which is useful for training due to computational matrix operation optimizations that quickly compute matrix operations. The matrices Q, K and V are defined as the matrices where the i th rows are vectors q_i, k_i, v_i respectively. Then we can represent the attention as $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

6.

Multimodality Transformers can also be used/adapted for modalities (input or output) beyond just text, usually by finding a way to "tokenize" the modality. Multimodal models can either be trained from scratch, or by finetuning. A 2022 study found that transformers pretrained only on natural language can be finetuned on only 0.03% of parameters and become competitive with LSTMs on a variety of logical and visual tasks, demonstrating transfer learning. The LLaVA was a vision-language model composed of a language model (Vicuna-13B) and a vision model (ViT-L/14), connected by a linear layer. Only the linear layer is finetuned. Vision transformers adapt the transformer to computer vision by breaking down input images as a series of patches, turning them into vectors, and treating them like embedding vector of tokens in a standard transformer. Conformer and later Whisper follow the same pattern for

speech recognition, first turning the speech signal into a spectrogram, which is then treated like an image, i.e. broken down into a series of patches, turned into vectors and treated like embedding vector of tokens in a standard transformer. Perceivers are a variant of transformers designed for multimodality. For image generation, notable architectures are DALL-E 1 (2021), Parti (2022), Phenaki (2023), and Muse (2023). Unlike later models, DALL-E is not a diffusion model. Instead, it uses a decoder-only transformer that autoregressively generates a text, followed by the token representation of an image, which is then converted by a variational autoencoder to an image. Parti is an encoder–decoder transformer, where the encoder processes a text prompt, and the decoder generates a token representation of an image. Muse is an encoder-only transformer that is trained to predict masked image tokens from unmasked image tokens. During generation, all input tokens are masked, and the highest-confidence predictions are included for the next iteration, until all tokens are predicted. Phenaki is a text-to-video model. It is a bidirectional masked transformer conditioned on pre-computed text tokens. The generated tokens are then decoded to a video.